Proyecto: CPU multiciclo S-MIPS en Logisim Evolution

Orientación y especificación

Arquitectura de Computadoras — Curso 2026–2027

Resumen

En este proyecto individual cada estudiante diseña e implementa, dentro de una placa de Logisim Evolution ya construida, una CPU *multiciclo* que ejecuta el repertorio S-MIPS estudiado en clase. Este documento es la especificación completa: dice qué hay que construir, qué está permitido y qué no, cómo se comprueba el funcionamiento, qué se entrega y cómo se evalúa. Las palabras *debe* y *no debe* indican requisitos obligatorios; *puede* indica libertad de diseño. Los requisitos se numeran [**R1**], [**R2**], ... para citarlos sin ambigüedad.

Índice

1. Objetivo y alcance **3**

2. Material entregado **3**

3. La placa y la interfaz de la CPU **3**
3.1. Circuitos de la placa . . . 3
3.2. Pines de la CPU . . . 4
3.3. Comportamiento temporal . . . 4
3.4. Mapa de memoria y valores iniciales . . . 4

4. El repertorio S-MIPS **5**
4.1. Estado arquitectónico . . . 5
4.2. Formatos . . . 5
4.3. Instrucciones . . . 5
4.4. Casos que fija este documento . . . 6
4.5. Servicios de `syscall` . . . 7

5. Requisitos de la microarquitectura **7**
5.1. CPU multiciclo . . . 7
5.2. Componentes obligatorios . . . 7
5.3. Uso de VHDL . . . 8
5.4. Componentes de Logisim permitidos . . . 9

6. Herramientas y flujo de trabajo **9**
6.1. Requisitos de software . . . 9
6.2. Abrir el circuito . . . 9
6.3. Ejecutar un programa a mano . . . 9
6.4. Comprobación automática . . . 9
6.5. Escribir programas propios . . . 10

7. Programas de prueba propios **10**

8. Entrega **11**

9. Evaluación **12**
9.1. Requisitos de admisión 12
9.2. Criterios 12

10. Organización **12**

11. Preguntas frecuentes **12**

1. Objetivo y alcance

El objetivo es construir una CPU que ejecute correctamente cualquier programa escrito con las instrucciones de S-MIPS (sección 4), cargado en la memoria de la placa, y que se comunique con el exterior mediante el terminal (TTY) y el LED *Halt*.

Se entrega resuelto todo lo que no es la CPU: el reloj, el botón de reset, la memoria, el terminal y el LED. El trabajo del estudiante es exclusivamente el contenido del subcircuito `CPU` y de los subcircuitos que este utilice.

Notación de bits. $X[i{:}j]$ designa los bits i a j de la señal X, ambos incluidos, con $i \geq j$; el bit 0 es el menos significativo. Por ejemplo, `MEM_ADDR[13:2]` son los 12 bits de la dirección que van del 13 al 2, y $X[k]$ es el bit k.

[R1] La CPU debe implementarse dentro del subcircuito `CPU` del archivo `circuit/smips.circ` entregado, usando Logisim Evolution **5.0.0**.

[R2] No deben modificarse los circuitos `Board` ni `Memory`. En el subcircuito `CPU` no deben cambiarse el nombre, el ancho ni el sentido de sus pines, ni borrarse y volver a crearse un pin, ni modificarse su apariencia (*Proyecto → Editar apariencia del circuito*). Los pines sí pueden moverse de sitio dentro del circuito `CPU`.

El motivo de **[R2]**: la `Board` se conecta a la CPU por la posición de los puertos en la apariencia del subcircuito (el rectángulo que se ve en la placa). Mover un pin dentro del circuito `CPU` no cambia esa apariencia: Logisim mantiene el vínculo entre el pin y su puerto. En cambio, editar la apariencia o borrar un pin y crearlo de nuevo (aunque tenga el mismo nombre) cambia la posición del puerto en el rectángulo, y la placa deja de estar conectada a ese pin.

2. Material entregado

Archivo o carpeta	Contenido
`circuit/smips.circ`	Placa con los circuitos `Board`, `Memory` y `CPU` (vacío, solo con pines).
`programs/`	Programas `.asm`: ejemplos (`hello`, `array_sum`, `gcd`, `multiply`) y pruebas de las instrucciones (`arithmetic`, `memory`, `branches`).
`images/`	Imágenes de memoria ya ensambladas de los programas de ejemplo.
`tools/assemble.py`	Ensambla un `.asm` con MARS y genera la imagen para la RAM.
`tools/run_tests.py`	Ejecuta programas en la CPU y compara la salida del TTY con la de MARS.
`open.sh`	Abre un circuito en Logisim; si tiene entidades VHDL, configura Questa y activa la simulación VHDL.
`README.md`	Resumen de la placa y del flujo de trabajo.

3. La placa y la interfaz de la CPU

3.1. Circuitos de la placa

- `Board` (circuito principal): reloj, botón *Reset*, la CPU, la memoria, el TTY y el LED *Halt*, conectados por túneles.
- `Memory`: una RAM de 4096 palabras de 32 bits (16 KiB) con escritura por bytes.
- `CPU`: el subcircuito a implementar.

3.2. Pines de la CPU

Pin	Sentido	Ancho	Significado
`CLK`	entrada	1	Reloj de la placa.
`RESET`	entrada	1	Vale 1 mientras el botón *Reset* está pulsado.
`MEM_RDATA`	entrada	32	Palabra de memoria que contiene la dirección `MEM_ADDR`.
`MEM_ADDR`	salida	32	Dirección de byte para leer o escribir.
`MEM_WDATA`	salida	32	Dato a escribir, colocado en sus carriles de byte.
`MEM_BE`	salida	4	Habilitación de escritura por byte.
`MEM_WE`	salida	1	Habilitación de escritura en memoria.
`TTY_DATA`	salida	7	Código ASCII del carácter a imprimir.
`TTY_WE`	salida	1	Habilitación de escritura en el TTY.
`HALT`	salida	1	Enciende el LED *Halt*: el programa terminó.

3.3. Comportamiento temporal

Todos los elementos de la placa que guardan estado actúan en el **flanco de subida** de `CLK`.

Lectura de memoria. Es combinacional: `MEM_RDATA` muestra, en el mismo ciclo, la palabra alineada que contiene la dirección `MEM_ADDR`. La memoria tiene $4096 = 2^{12}$ palabras y elige la palabra con los 12 bits `MEM_ADDR[13:2]`. Ignora `MEM_ADDR[1:0]`, que indican el byte dentro de la palabra, e ignora `MEM_ADDR[31:14]`, de modo que las direcciones se repiten cada 16 KiB (por ejemplo, `0x4000` es la misma posición que `0x0000`).

Escritura en memoria. Si `MEM_WE` $= 1$ en un flanco de subida, en la palabra `MEM_ADDR[13:2]` se escriben los bytes k con `MEM_BE[`k`]` $= 1$, tomando el byte k de `MEM_WDATA` (bits $8k + 7..8k$). Los bytes con `MEM_BE[`k`]` $= 0$ no cambian. Con `MEM_WE` $= 0$ no se escribe nada, cualquiera que sea `MEM_BE`.

Orden de los bytes. *Little-endian*: el byte de la dirección $4n + k$ ($k \in \{0, 1, 2, 3\}$) ocupa los bits $8k + 7..8k$ de la palabra n. Por tanto, `sw` usa `MEM_BE` $=$ `1111`; `sb` en la dirección d debe activar solo el bit d[1:0] de `MEM_BE` y colocar el byte en el carril d[1:0] de `MEM_WDATA`; `lb` y `lbu` deben extraer el byte d[1:0] de `MEM_RDATA` y extenderlo. Este trabajo corresponde a la CPU.

TTY. Si `TTY_WE` $= 1$ en un flanco de subida, el TTY imprime el carácter `TTY_DATA`. El código 10 (`\n`) salta de línea. El botón *Reset* borra el TTY.

Halt. El LED muestra el valor de `HALT`.

[R3] Todos los elementos secuenciales de la CPU deben actualizarse en el flanco de subida de `CLK`, conectados directamente a `CLK`. No deben generarse relojes derivados (ninguna lógica en el camino del reloj).

[R4] `TTY_WE` debe valer 1 durante **exactamente un** flanco de subida por cada carácter impreso, y 0 en cualquier otro momento.

[R5] `MEM_WE` debe valer 1 solo en el ciclo en que se ejecuta la escritura de un `sw` o un `sb`.

3.4. Mapa de memoria y valores iniciales

El mapa de memoria corresponde a la configuración *Compact, Text at Address 0* de MARS (*Settings* → *Memory Configuration*).

Elemento	Dirección o valor	Nota
Segmento `.text`	`0x0000`	primera instrucción del programa
Segmento `.data`	`0x2000`	datos estáticos
`$gp` inicial	`0x1800`	
`$sp` inicial	`0x3FFC`	la pila crece hacia direcciones menores
Tamaño de la memoria	16 KiB (`0x0000`–`0x3FFF`)	

[R6] Mientras `RESET` = 1, la CPU debe quedar en su estado inicial: PC = 0, `$gp` = `0x1800`, `$sp` = `0x3FFC`, los otros 30 registros a 0 y la unidad de control en su estado inicial, con `MEM_WE`, `TTY_WE` y `HALT` a 0. Al pasar `RESET` a 0, la CPU debe comenzar a ejecutar la instrucción de la dirección `0x0000`. El reset puede ser asíncrono (entradas de borrado de los registros) o síncrono.

[R7] El reset no debe modificar la memoria. El programa se carga en la RAM antes de pulsar *Reset*.

4. El repertorio S-MIPS

Esta sección resume el repertorio estudiado en clase y fija los casos que el material no cubre.

4.1. Estado arquitectónico

32 registros de propósito general de 32 bits (`$0`–`$31`). El registro `$0` (`$zero`) vale siempre 0: leerlo da 0 y escribir en él no tiene efecto. El PC (32 bits) no pertenece al banco de registros. No hay registro de banderas ni registros Hi/Lo.

4.2. Formatos

	bits 31–26	25–21	20–16	15–11	10–6	5–0
R	op	rs	rt	rd	shamt	funct
I	op	rs	rt	inmediato (16)		
J	op	dirección (26)				

Notación: $R[x]$ es el registro x; $M_{32}[d]$ y $M_8[d]$ son la palabra y el byte de la dirección d; SignExt y ZeroExt extienden 16 bits a 32 con el bit 15 o con ceros; $\{a, b\}$ es la concatenación. Salvo que se indique otra cosa, PC $\leftarrow$ PC + 4. En las instrucciones de memoria, $d = R[rs] + \text{SignExt}(imm)$. Los códigos op y funct están en hexadecimal.

4.3. Instrucciones

Instrucción	Fmt	op	funct	Semántica
Aritméticas y lógicas				
`add rd, rs, rt`	R	00	20	$R[rd] \leftarrow R[rs] + R[rt]$
`sub rd, rs, rt`	R	00	22	$R[rd] \leftarrow R[rs] - R[rt]$
`and rd, rs, rt`	R	00	24	$R[rd] \leftarrow R[rs]$ & $R[rt]$
`or rd, rs, rt`	R	00	25	$R[rd] \leftarrow R[rs] \mid R[rt]$
`slt rd, rs, rt`	R	00	2A	$R[rd] \leftarrow (R[rs] <_s R[rt])$? 1 : 0 (con signo)
`sll rd, rt, shamt`	R	00	00	$R[rd] \leftarrow R[rt] \ll shamt$
`srl rd, rt, shamt`	R	00	02	$R[rd] \leftarrow R[rt] \gg_l shamt$ (entran ceros)
Con inmediato				
`addi rt, rs, imm`	I	08	—	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$

Instrucción	Fmt	op	funct	Semántica
`slti rt, rs, imm`	I	0A	—	$R[rt] \leftarrow (R[rs] <_s \text{SignExt}(imm))\ ?\ 1 : 0$
`andi rt, rs, imm`	I	0C	—	$R[rt] \leftarrow R[rs]$ & ZeroExt(imm)
`ori rt, rs, imm`	I	0D	—	$R[rt] \leftarrow R[rs]$ | ZeroExt(imm)
`lui rt, imm`	I	0F	—	$R[rt] \leftarrow \{imm, \texttt{0x0000}\}$
Acceso a memoria				
`lw rt, imm(rs)`	I	23	—	$R[rt] \leftarrow M_{32}[d]$
`sw rt, imm(rs)`	I	2B	—	$M_{32}[d] \leftarrow R[rt]$
`lb rt, imm(rs)`	I	20	—	$R[rt] \leftarrow \text{SignExt}(M_8[d])$
`lbu rt, imm(rs)`	I	24	—	$R[rt] \leftarrow \text{ZeroExt}(M_8[d])$
`sb rt, imm(rs)`	I	28	—	$M_8[d] \leftarrow R[rt][7{:}0]$
Control de flujo				
`beq rs, rt, etiq`	I	04	—	si $R[rs] = R[rt]$: PC $\leftarrow$ PC + 4 + (SignExt(imm) $\ll 2$)
`bne rs, rt, etiq`	I	05	—	si $R[rs] \neq R[rt]$: PC $\leftarrow$ PC + 4 + (SignExt(imm) $\ll 2$)
`j etiq`	J	02	—	PC $\leftarrow \{(\text{PC}+4)[31{:}28], dir, 00\}$
`jal etiq`	J	03	—	$R[31] \leftarrow$ PC + 4; PC $\leftarrow \{(\text{PC}+4)[31{:}28], dir, 00\}$
`jr rs`	R	00	08	PC $\leftarrow R[rs]$
Entorno				
`syscall`	R	00	0C	servicio indicado por `$v0` (sección 4.5)
*Generadas por MARS (obligatorias, ver **[R9]**)*				
`addu rd, rs, rt`	R	00	21	igual que `add`
`addiu rt, rs, imm`	I	09	—	igual que `addi` (extensión con signo)

4.4. Casos que fija este documento

[R8] La CPU debe ejecutar las 23 instrucciones de S-MIPS con la semántica de la tabla anterior.

[R9] La CPU debe ejecutar `addu` exactamente como `add` y `addiu` exactamente como `addi`. Motivo: MARS traduce las pseudoinstrucciones `move` y `li` a `addu` y `addiu`, que no pertenecen a S-MIPS.

[R10] Una palabra cuyo par (op, funct) no aparece en la tabla es una **instrucción ilegal**: la CPU debe detenerse como con `syscall` 10 (`HALT` = 1, sin modificar ningún estado). La palabra `0x00000000` es `sll $zero, $zero, 0` (`nop`): es legal y no tiene efecto.

[R11] La instrucción se identifica solo por op y, si op = 0, por funct. No es obligatorio comprobar que los campos no usados valgan 0.

Situaciones que los programas de prueba **nunca** producen y cuyo comportamiento, por tanto, queda libre: accesos a palabra no alineados (`lw`/`sw` con $d[1{:}0] \neq 0$), saltos a direcciones no múltiplo de 4, desbordamiento aritmético en `add`/`addi`/`sub` (S-MIPS no tiene excepciones: el resultado es el de la suma módulo 2^{32}, pero MARS detiene el programa, así que no se usará) y accesos fuera de los 16 KiB.

4.5. Servicios de `syscall`

`$v0`	Servicio	Efecto que debe producir la CPU
11	imprimir carácter	`TTY_DATA` ← `$a0[6:0]` con `TTY_WE` = 1 durante un flanco ([**R4**]); después continúa con la instrucción siguiente.
10	terminar	`HALT` ← 1. A partir de ese momento la CPU no debe modificar registros, PC ni memoria, ni escribir en el TTY, y `HALT` debe permanecer en 1 hasta el siguiente reset.
otro	—	Sin efecto: la CPU continúa con la instrucción siguiente.

Los servicios 1 (imprimir entero), 4 (imprimir cadena) y 5 (leer entero) de MARS **no** se implementan en hardware. Los programas los resuelven en software con el servicio 11; `programs/array_sum.asm` muestra cómo.

5. Requisitos de la microarquitectura

5.1. CPU multiciclo

[**R12**]
La CPU debe ser **multiciclo**: cada instrucción se ejecuta en varios ciclos de reloj, una sola instrucción a la vez (no se permite segmentación ni ejecutar dos instrucciones en paralelo). El número de ciclos puede variar según la instrucción.

[**R13**]
Se usa la única memoria de la placa para instrucciones y datos. La instrucción en curso debe guardarse en un registro de instrucción (IR) durante todos sus ciclos.

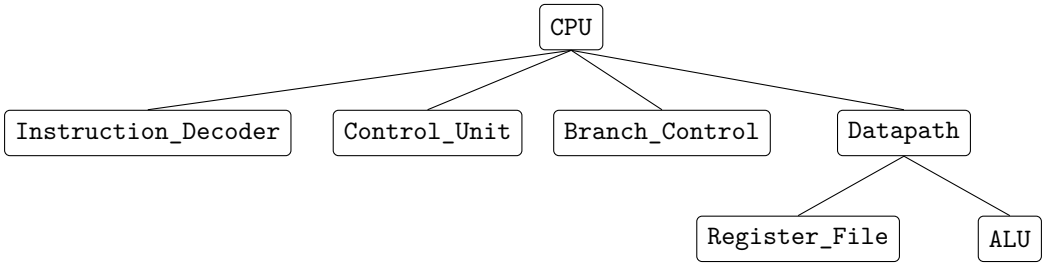
[**R14**]
La ruta de datos debe tener **una sola ALU** y ningún otro sumador: el cálculo de $\mathrm{PC} + 4$, de los destinos de salto y de las direcciones de memoria se hace con esa ALU en ciclos distintos. Se permite un comparador de igualdad en `Branch_Control` como alternativa a la salida *zero* de la ALU.

[**R15**]
La secuencia de ciclos la gobierna una máquina de estados síncrona dentro de `Control_Unit`, construida con biestables o registros (el estado) y lógica combinacional (próximo estado y salidas). Puede ser de tipo Moore o Mealy, y la codificación de los estados es libre.

No se exige un número máximo de ciclos por instrucción. Como referencia, una implementación de entre 3 y 5 ciclos por instrucción termina el programa de prueba más largo en unos 8000 ciclos; `run_tests.py` abandona un programa a los 60 000 ciclos.

5.2. Componentes obligatorios

La CPU debe organizarse en los siguientes subcircuitos, con **exactamente estos nombres** y esta jerarquía:



Subcircuito	Función mínima obligatoria
`Instruction_Decoder`	Combinacional, sin estado. Entradas: los campos op y funct de la instrucción. Salidas: la información que necesita la unidad de control (clase de instrucción, operación de la ALU, tipo de extensión, etc.; el conjunto de salidas es libre). Detecta las instrucciones ilegales ([**R10**]).
`Control_Unit`	La máquina de estados de [**R15**]. A partir del estado y de las salidas del decodificador genera todas las señales de control de la ruta de datos (selectores de multiplexores y habilitaciones de escritura), además de `MEM_WE`, `TTY_WE` y `HALT`.
`Branch_Control`	Combinacional. Decide si se escribe el PC en el ciclo actual a partir de las señales de control y del resultado de la comparación de `beq`/`bne`.
`Datapath`	PC, IR, registros intermedios, multiplexores, extensión del inmediato, alineación de bytes para `lb`/`lbu`/`sb` y cálculo del destino de `j`/`jal`. Contiene `Register_File` y `ALU`.
`Register_File`	32 registros de 32 bits; dos puertos de lectura combinacionales y un puerto de escritura que actúa en el flanco de subida de `CLK` cuando su habilitación vale 1; `$zero` siempre 0; valores de reset de [**R6**].
`ALU`	Combinacional. Dos operandos de 32 bits y un código de operación; resultado de 32 bits y salida *zero* (1 si el resultado es 0). Debe implementar al menos suma, resta, AND, OR, comparación con signo (SLT) y desplazamientos lógicos a izquierda y derecha. La codificación de las operaciones es libre.

Se pueden crear subcircuitos adicionales con cualquier nombre (por ejemplo `Immediate_Extender` o `Memory_Interface`) en cualquier nivel de la jerarquía.

[**R16**]

Deben existir los seis subcircuitos anteriores, con esos nombres y esa jerarquía, y cada uno debe cumplir su función mínima.

[**R17**]

Dentro de la CPU no se permiten los componentes RAM ni ROM de Logisim. En particular, si `Register_File` se implementa como circuito, debe construirse con registros o biestables, multiplexores y decodificadores.

5.3. Uso de VHDL

[**R18**]

A lo sumo dos de los tres subcircuitos `Register_File`, `ALU` e `Instruction_Decoder`, a elección del estudiante, pueden implementarse como entidades VHDL de Logisim (ninguno, uno o dos). Los demás, y todo el resto de la CPU, deben ser circuitos de Logisim. No puede haber ningún otro código VHDL en la CPU.

[**R19**]

Cada entidad VHDL debe llamarse como el subcircuito que sustituye (`Register_File`, `ALU` o `Instruction_Decoder`; VHDL no distingue mayúsculas). Sus puertos deben ser solo de tipo `std_logic` o `std_logic_vector`, sin *generics*. Puede usar únicamente las bibliotecas `ieee.std_logic_1164` e `ieee.numeric_std`, y debe compilar como VHDL-2002 sin errores ni avisos.

[**R20**]

Las entidades VHDL deben ser descriptivas y sintetizables: no se permiten `wait`, retardos (`after`), archivos ni instrucciones de simulación.

En Logisim, una entidad se crea desde el menú *Proyecto* (*Añadir entidad VHDL*) y se coloca como cualquier otro componente. Para simularla hace falta Questa: `open.sh` la configura (sección 6). Si no se usa VHDL, Questa no es necesaria.

5.4. Componentes de Logisim permitidos

Dentro de la CPU pueden usarse los componentes de las bibliotecas *Cableado* (incluidos túneles, separadores, constantes y extensores de bits), *Puertas*, *Multiplexores* (multiplexores, demultiplexores, decodificadores, codificadores de prioridad), *Aritmética* (sumador, restador, comparador, desplazador, negador) y *Memoria* salvo RAM y ROM (biestables, registros y contadores). Siempre respetando **[R14]** (una sola ALU, ningún otro sumador) y **[R17]**.

6. Herramientas y flujo de trabajo

6.1. Requisitos de software

- Java 21 o superior.
- Logisim Evolution **5.0.0** (variable `LOGISIM_JAR` con la ruta al `.jar`).
- MARS 4.5 (variable `MARS_JAR`; por defecto `~/Apps/mars/Mars4_5.jar`).
- Solo si se usa VHDL (**[R18]**): Questa Intel/Altera FPGA Starter Edition con su licencia gratuita (variables `QUESTA_BIN` con la carpeta `bin` y `SALT_LICENSE_SERVER` con el archivo de licencia).
- Python 3 y una sesión gráfica.

6.2. Abrir el circuito

```
bash open.sh # abre circuit/smips.circ (y activa la simulacion VHDL si hace falta)
```

6.3. Ejecutar un programa a mano

1. Ensamblar el programa; por ejemplo, para `programs/hello.asm`:

   ```
   python3 tools/assemble.py programs/hello.asm # genera images/hello.hex
   ```

   El ensamblador avisa si el código contiene instrucciones que la CPU tratará como ilegales.
2. En Logisim, entrar en `Memory`, clic derecho sobre la RAM → *Cargar imagen…* y elegir el `.hex`.
3. Volver a `Board`, pulsar *Reset* con la herramienta de mano y activar el reloj (*Simular* → *Ticks habilitados*).
4. El texto aparece en el TTY; al terminar se enciende *Halt*.

6.4. Comprobación automática

```
python3 tools/run_tests.py # programs/*.asm y tests/*.asm
python3 tools/run_tests.py programs/memory.asm # un solo programa
python3 tools/run_tests.py --circuito otra.circ # otro archivo de circuito
```

Para cada programa, `run_tests.py` obtiene la salida esperada ejecutándolo en MARS, lo ejecuta en la placa, espera a que `HALT` valga 1 y compara el texto del TTY con el de MARS (se ignoran los espacios al final de cada línea). Un programa es **correcto** si la CPU se detiene antes de 60 000 ciclos y el texto coincide. Con entidades VHDL la simulación se hace junto con Questa y es lenta (decenas de ciclos por segundo): la batería completa tarda unos minutos.

6.5. Escribir programas propios

Los programas deben usar solo instrucciones de S-MIPS y las pseudoinstrucciones vistas en clase (`move`, `li`, `la`, `blt`, `bge`, `bgt`, `ble`, `nop`), que MARS traduce a instrucciones de S-MIPS o a `addu`/`addiu`. Para la salida solo están los servicios 10 y 11; el TTY tiene 12 líneas de 48 caracteres.

7. Programas de prueba propios

Además de la CPU, el estudiante escribe sus propios programas en ensamblador S-MIPS para probarla. Los programas de `programs/` entregados con el proyecto no cuentan como propios.

[R21]

El repositorio debe incluir una carpeta `tests/` con programas `.asm` escritos por el estudiante que, entre todos, ejecuten **todas las instrucciones** de la tabla de la sección 4 (las 23 de S-MIPS, más `addu` y `addiu`) en **todos los casos** de la tabla 3. Cada programa debe cumplir:

1. Usar solo lo permitido en la sección 6.5: instrucciones de S-MIPS, las pseudoinstrucciones vistas en clase, y los servicios 11 y 10 (termina con `syscall` 10).
2. Imprimir los **valores calculados** (por ejemplo, en hexadecimal o en decimal), no solo un mensaje fijo: si una instrucción falla, la salida debe ser distinta.
3. Producir exactamente la misma salida en MARS y en la CPU del estudiante, comprobado con `run_tests.py`.
4. Caber en el TTY: como máximo 12 líneas de 48 caracteres.
5. Empezar con un comentario que enumere las instrucciones y los casos de la tabla 3 que cubre.

Tabla 3: Casos que deben ejecutar los programas de `tests/`.

Instrucciones	Casos obligatorios
`add`, `sub`	Con operandos positivos y con al menos un operando negativo.
`and`, `or`	Con operandos que tengan unos en los 16 bits altos.
`slt`, `slti`	Un caso con resultado 1 y otro con resultado 0; al menos uno con un operando negativo en el que la comparación con signo y sin signo den resultados distintos.
`sll`, `srl`	Con `shamt` $= 0$ y con `shamt` $\neq 0$; `srl` sobre un valor negativo (deben entrar ceros).
`addi`	Con un inmediato negativo.
`addiu`, `addu`	Mediante las pseudoinstrucciones `li` (constante entre -2^{15} y $2^{15}-1$) y `move`.
`andi`, `ori`	Con un inmediato cuyo bit 15 vale 1 (debe extenderse con ceros).
`lui`	Con un inmediato cuyo bit 15 vale 1.
`lw`, `sw`	Con desplazamiento positivo y con desplazamiento negativo.
`lb`, `lbu`	Sobre un byte cuyo bit 7 vale 1, en al menos dos posiciones distintas dentro de la palabra (d[1:0] distinto).
`sb`	En las cuatro posiciones de una misma palabra ($d[1{:}0] = 0, 1, 2, 3$), comprobando después con `lw` que los otros bytes no cambiaron.
`beq`, `bne`	Cada uno tomado y no tomado; al menos un salto hacia atrás.
`j`	Al menos una vez.

Instrucciones	Casos obligatorios
`jal`, `jr`	Un procedimiento que llama a otro (no hoja), guardando `$ra` en la pila y retornando correctamente.
`syscall`	Servicios 11 y 10.
`$zero`	Una instrucción que escribe en `$zero`, comprobando después que sigue valiendo 0.

El informe incluye una tabla que indica, para cada fila de la tabla 3, en qué programa de `tests/` se cubre.

8. Entrega

[**R22**]

La entrega es un **enlace a un repositorio de GitHub**, enviado al profesor por **Telegram** antes de la fecha límite, que se comunicará posteriormente. Si el repositorio es privado, el estudiante debe dar acceso de lectura a la cuenta de GitHub que indique el profesor junto con la fecha. Se evalúa el último *commit* anterior a la fecha límite.

[**R23**]

El repositorio debe contener, en estas rutas:

1. `circuit/smips.circ`: el circuito con la CPU implementada ([**R1**], [**R2**]).
2. `circuit/`*entidad*`.vhd`: un archivo por cada entidad VHDL usada (`register_file.vhd`, `alu.vhd` o `instruction_decoder.vhd`), idéntico al código incrustado en `smips.circ`. Si no se usa VHDL, no hay ninguno.
3. `informe.pdf`: informe de 10 páginas como máximo.
4. `tests/`: los programas de prueba propios ([**R21**]).
5. `salida-tests.txt`: la salida completa de `python3 tools/run_tests.py` sobre la versión entregada (incluye `programs/` y `tests/`).

Puede contener además el resto del material del proyecto.

El informe debe incluir, en este orden:

1. Nombre del estudiante y qué componentes se implementaron en VHDL (si alguno), con una breve justificación.
2. Diagrama de la ruta de datos con los nombres de las señales de control.
3. Diagrama o tabla de la máquina de estados: estados, transiciones y condiciones.
4. Tabla con el valor de cada señal de control en cada estado.
5. Codificación de las operaciones de la ALU y de las salidas del decodificador.
6. Número de ciclos de cada clase de instrucción.
7. Tabla de cobertura de los programas de `tests/` (sección 7).
8. Resultados de `run_tests.py` y limitaciones o errores conocidos. Un error declarado se penaliza menos que uno oculto.

9. Evaluación

9.1. Requisitos de admisión

Una entrega **no se evalúa** (calificación 0) si incumple [**R1**] o [**R2**], si el circuito no abre en Logisim Evolution 5.0.0, o si incumple [**R18**] (más de dos componentes en VHDL, o VHDL en componentes no permitidos).

9.2. Criterios

Criterio	Cómo se mide	Peso
Corrección	Programas de `programs/`, más programas no publicados del profesor que usan solo S-MIPS (incluidas `addu`/`addiu`) y los servicios 10 y 11. Cada programa correcto suma por igual.	40 %
Pruebas propias	Programas de `tests/`: cubren todas las filas de la tabla 3, cumplen [**R21**] y dan la misma salida en MARS y en la CPU. Cada fila cubierta suma por igual.	10 %
Estructura	Cumplimiento de [**R3**]–[**R7**] y [**R12**]–[**R20**]: componentes obligatorios con su función, multiciclo con una sola ALU, máquina de estados, reglas de VHDL.	20 %
Claridad del circuito	Nombres de señales y componentes que se entienden, organización en subcircuitos, textos explicativos en Logisim.	10 %
Informe	Completo según la sección anterior y coherente con el circuito entregado.	10 %
Defensa	El estudiante explica, sobre el circuito, cualquier parte de la CPU que se le pregunte.	10 %

10. Organización

- El proyecto es **individual**.
- La fecha límite de entrega y la de la defensa se comunicarán posteriormente.
- La entrega se hace por Telegram con un enlace a GitHub ([**R22**]).

11. Preguntas frecuentes

¿Puedo añadir pines a la CPU o cambiar la placa? No ([**R2**]). Toda la comunicación pasa por los diez pines de la sección 3.

¿Puedo mover los pines dentro del circuito CPU? Sí. Lo que no se puede es editar la apariencia ni borrar y recrear un pin ([**R2**]).

¿Puedo usar túneles dentro de la CPU? Sí. Se recomiendan, con nombres que digan qué señal llevan.

¿Puedo usar un sumador aparte para $\mathrm{PC} + 4$**?** No ([**R14**]). Ese cálculo se hace con la ALU, en un ciclo en que esta no se usa para otra cosa.

¿El reset debe ser síncrono o asíncrono? Cualquiera de los dos, siempre que se cumpla [**R6**]. ([**R17**]).

¿Dónde va la extensión de signo de `lb`? En la ruta de datos (puede ser un subcircuito propio). La memoria siempre devuelve la palabra completa.

¿Qué hago con `li` y `move`? Nada especial en los programas: la CPU debe aceptar `addiu` y `addu` ([**R9**]).

¿Qué ocurre con `syscall` 1, 4 o 5? No tienen efecto en la CPU (sección 4.5). Si un programa los usa, su salida no coincidirá con la de MARS; por eso los programas de prueba no los usan.

¿Puedo hacer en VHDL la ALU, el decodificador y el banco de registros? No: como mucho dos de los tres ([**R18**]). Tampoco es obligatorio usar VHDL.

¿Cuenta la velocidad de la CPU? No, mientras termine cada programa antes de 60 000 ciclos.